

Machine Learning Force Fields: Predicting Potential Energy and Atomic Forces of Molecular Structures

Amirarsalan Sanati

amirarsalan.sanati.001@student.uni.lu

University of Luxembourg

Luxembourg

Abstract

Molecular simulations are central to chemistry and materials science for probing mechanisms, designing drugs and materials, and accessing systems that are hard to study experimentally. Achieving quantum-chemical accuracy is, however, often prohibitively expensive because evaluating forces with first-principles electronic-structure methods dominates the cost. Machine-learning force fields mitigate this by learning the mapping from atomic structure to potential energy and forces from data. In this work we study how to make graph neural network (GNN) force fields more accurate without making them much larger. Building on SchNet, we propose a GNN that only changes how local atomic environments are aggregated, yielding two variants whose parameter counts increase by a constant factor of roughly $1.5\text{--}2\times$. Despite this modest growth, our models reduce test force and energy errors by factors of $3\text{--}10$ on the rMD17 benchmark—about a **90%** reduction in total loss compared to SchNet—and converge faster in terms of epochs. This shows that revisiting aggregation in GNNs can yield substantial practical gains for molecular simulation and make high-accuracy force fields more accessible.

1 Introduction

The goal of computational chemistry and materials science is to relate the *jiggings and wiggings* of atoms to macroscopic behaviour [8, 16]. This is formalized by the **Potential Energy Surface (PES)**, a high-dimensional map from molecular geometries to potential energies [16], which underlies **molecular dynamics (MD)** simulations of processes such as protein folding and chemical reactions [6, 14].

Classical approaches face a strong accuracy–cost trade-off [8]. *Ab initio* methods like **Density Functional Theory (DFT)** are accurate but typically limited to systems of a few hundred atoms [16], whereas *molecular mechanics (MM)* scales to millions of atoms but often misses electronic effects like bond breaking, polarization, and charge transfer [6, 16]. **Machine Learning Force Fields (MLFFs)** bridge this gap by fitting high-quality quantum data and recovering near-*ab initio* accuracy at much lower cost [1, 6, 16], enabling applications in drug discovery, battery design, and electrochemistry [6, 10].

Here we ask how far one can push MLFF accuracy by modifying only the GNN architecture, without greatly increasing model size or runtime. We propose a SchNet-based GNN that changes how information from neighboring atoms is aggregated, yielding two variants whose parameter counts grow by only about $1.5\text{--}2\times$ compared to the original model. On the rMD17 benchmark these architectures cut total test error by up to 90%, corresponding to

$3\text{--}10\times$ lower energy and force mean absolute errors, and they converge faster, indicating that improved aggregation alone can substantially enhance the usefulness of MLFFs for molecular simulation.

2 Related Work

Early MLFFs used handcrafted geometric descriptors such as Atom-Centered Symmetry Functions and SOAP [1, 16], combined with kernel methods like GAP and sGDML [3, 4, 16]. While sGDML is highly data-efficient for small molecules, its global descriptors do not scale well to large or extended systems [16].

Graph Neural Networks (GNNs) model atoms as nodes and interactions as edges. SchNet introduced continuous-filter convolutions based on interatomic distances [15], and DimeNet++ added directional message passing with bond angles to better capture non-equilibrium structures and reactions [9]. Modern MLFFs are dominated by equivariant architectures that ensure forces and other tensors transform correctly under rotations: NequIP showed that $E(3)$ -equivariant convolutions greatly improve data efficiency [2], PaiNN provided efficient equivariant message passing for scalar and tensorial properties [14], Allegro scaled local equivariant models to over 10^8 atoms [12], and MACE achieved top accuracy with higher-order messages [1]. Further variants such as ViSNet, EquiformerV2, and SO3krates explore vector–scalar interactions, Transformer-style attention, and spherical self-attention to model complex geometry and long-range correlations [8, 11, 18].

Practical deployment is supported by frameworks like TorchMD-Net, which integrates MLFFs into standard MD packages [13], and by tools such as MAPLE and FFAST for automated enzyme and reaction modeling and for diagnosing problematic molecular configurations [7, 17].

3 Experiments

3.1 Dataset and task

We evaluate on the revised MD17 (rMD17) dataset [5], a standard benchmark of molecular dynamics trajectories where each configuration is labeled with *ab initio* potential energy and atomic forces. In this work we use a single molecule (aspirin) and follow the dataset authors’ recommendation to restrict training to at most 1000 structures to avoid overfitting to temporally correlated MD frames. We perform a random split into 1000 training, 200 validation, and 1000 test configurations, and reuse the same split for all models and all hyperparameter settings. The target is to jointly predict energies and forces for each configuration.

3.2 Architectures

As baseline we implement SchNet [15] using continuous-filter convolutions and atom-wise energy prediction. Given atomic numbers Z_i , positions $\mathbf{r}_i$, and a learned embedding $\mathbf{h}_i^{(0)}$, each interaction block performs

$$m_{ij} = \phi_{\text{msg}}(\mathbf{h}_j) \odot W(r_{ij}), \quad r_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|, \quad (1)$$

$$\mathbf{m}_i = \sum_{j \in \mathcal{N}(i)} m_{ij}, \quad (2)$$

$$\mathbf{h}_i^{\text{new}} = \mathbf{h}_i + \phi_{\text{upd}}(\mathbf{m}_i), \quad (3)$$

where $W(r_{ij})$ is a distance filter obtained by applying an MLP to a Gaussian radial basis expansion of r_{ij} , ϕ_{msg} is a small MLP acting on the embeddings of the current interaction block, and ϕ_{upd} is a small residual MLP. Furthermore, ϕ_{msg} and ϕ_{upd} use *softplus* activation function. Softplus is a smooth, differentiable approximation of the ReLU (Rectified Linear Unit) function:

$$\text{Softplus}(x) = \log(1 + e^x) \quad (4)$$

After multiple interaction steps, an atom-wise network predicts per-atom energies, which are summed to obtain the total energy $E(\mathbf{R})$.

SchNet+. SchNet+ keeps the same message function m_{ij} but introduces multiple permutation-invariant aggregations over the incoming edge set of each atom:

$$\mathbf{m}_i^{\text{sum}} = \sum_j m_{ij}, \quad \mathbf{m}_i^{\text{avg}} = \frac{1}{|\mathcal{N}(i)|} \sum_j m_{ij}, \quad (5)$$

$$\mathbf{m}_i^{\text{max}} = \max_j m_{ij}, \quad \mathbf{m}_i^{\text{min}} = \min_j m_{ij}. \quad (6)$$

Each aggregated message is passed through the *same* update network, $\mathbf{h}_i^k = \mathbf{h}_i + \phi_{\text{upd}}(\mathbf{m}_i^k)$ for $k \in \{\text{sum}, \text{avg}, \text{max}, \text{min}\}$, producing four candidate node states. These are concatenated and linearly fused,

$$\mathbf{h}_i^{\text{new}} = W_f[\mathbf{h}_i^{\text{sum}}, \mathbf{h}_i^{\text{avg}}, \mathbf{h}_i^{\text{max}}, \mathbf{h}_i^{\text{min}}], \quad (7)$$

so the model can learn how much to trust each “view” of the neighborhood while adding only a constant-factor number of parameters.

SchNet++. SchNet++ trades the hand-crafted set of aggregations for a learnable generalized mean. Besides the standard sum aggregation, it computes a log-sum-exp f -mean over messages for each feature dimension:

$$(\mathbf{m}_i^{\text{lse}})_d = \frac{1}{\alpha_d} \log \left(\frac{1}{|\mathcal{N}(i)|} \sum_j \exp(\alpha_d (m_{ij})_d) \right), \quad (8)$$

where α_d is a learned per-feature sharpness parameter. As $\alpha_d \rightarrow 0$, this reduces to the arithmetic mean; large positive (negative) values interpolate smoothly towards max (min) pooling. Thisway, the model can still have the different “view”s (average, min, max pooling), while using much less number of parameters for the linear fusion. The two aggregates $\mathbf{m}_i^{\text{sum}}$ and $\mathbf{m}_i^{\text{lse}}$ are each passed through the shared update network and then concatenated and linearly fused:

$$\mathbf{h}_i^{\text{new}} = W_f[\mathbf{h}_i^{\text{sum}}, \mathbf{h}_i^{\text{lse}}]. \quad (9)$$

Both SchNet+ and SchNet++ preserve permutation invariance over atoms and neighbors, which is an important property required from MLFFs.

3.3 Training and evaluation

All models use 4 interaction blocks, hidden dimension $n = 128$, and $n_{\text{rbf}} = 50$ radial basis functions. Energies and forces are predicted jointly; forces are obtained as $\mathbf{F}_i = -\nabla_{\mathbf{r}_i} E(\mathbf{R})$ using automatic differentiation. The loss is a weighted sum of energy and force mean absolute error (MAE),

$$\mathcal{L} = w_E \text{MAE}(E, \hat{E}) + w_F \text{MAE}(\mathbf{F}, \hat{\mathbf{F}}), \quad (10)$$

with $w_E = 1$ and $w_F = 100$, following common practice to emphasize accurate forces. We train with AdamW for 100 epochs and explore a 2×2 grid of learning rates $\{1 \times 10^{-4}, 5 \times 10^{-4}\}$ and weight decays $\{10^{-4}, 10^{-2}\}$. For each configuration we record training and validation losses, test MAEs, and parameter counts; the best checkpoint is selected by validation loss. The full configuration and code are provided in the appendix to enable replication (see C).

Across all four optimizer settings, SchNet+ and SchNet++ consistently outperform the SchNet baseline on test energy and force MAE. Typical results show force MAE reductions from ≈ 20 –30 (SchNet) to ≈ 6 –10 (SchNet+ and SchNet++), and energy MAE reductions by one order of magnitude (see A for detailed results). Overall, total test loss decreases by roughly 90% relative to SchNet, while parameter counts increase only by factors of $\approx 2 \times$ (SchNet+) and $\approx 1.5 \times$ (SchNet++) (see B for mathematical analysis on number of parameters and their ratios). Learning curves indicate that the proposed models also converge faster in terms of epochs (see A).

4 Discussion

Our results suggest that the way a GNN aggregates information from neighboring atoms is a crucial lever for improving machine-learned force fields. By enriching SchNet’s sum aggregation with additional permutation-invariant statistics (sum/mean/min/max in SchNet+, and a learnable log-sum-exp mean in SchNet++), we obtain substantially better accuracy and faster convergence for a modest, constant-factor increase in model size. In other words, we improve the *quality* of each message-passing layer rather than simply stacking more layers or widening the network.

From a broader perspective, this study reinforces two insights. First, for data-limited settings such as rMD17, carefully designed inductive biases—here, multi-view neighborhood summaries—can be more effective than brute-force scaling in parameters. Second, generalized f -means provide a principled way to interpolate between classical aggregations like mean and max, yielding parameter-efficient gains (SchNet++) that complement the stronger but heavier multi-aggregation scheme (SchNet+). While our evaluation is restricted to a single molecule and a small hyperparameter grid, the consistency of the improvements suggests that aggregation design deserves more attention in future GNN-based MLFF architectures, and motivates follow-up work on extending these ideas to larger and more diverse molecular systems.

References

- [1] Ilyes Batatia, Gábor Csányi, David P. Kovács, Christoph Ortner, and Gregor Simm. 2022. MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields. (2022), 11423–11436. doi:10.52202/068431-0830
- [2] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P. Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E. Smidt, and Boris Kozinsky. 2022. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications* 13, 1 (04 May 2022), 2453.
- [3] Stefan Chmiela, Huziel E. Sauceda, Klaus-Robert Müller, and Alexandre Tkatchenko. 2018. Towards exact molecular dynamics simulations with machine-learned force fields. *Nature Communications* 9, 1 (24 Sep 2018), 3887. doi:10.1038/s41467-018-06169-2
- [4] Stefan Chmiela, Huziel E. Sauceda, Igor Poltavsky, Klaus-Robert Müller, and Alexandre Tkatchenko. 2019. sGDML: Constructing accurate and data efficient molecular force fields using machine learning. *Computer Physics Communications* 240 (2019), 38–45. doi:10.1016/j.cpc.2019.02.007
- [5] Anders S. Christensen and Anatole von Lilienfeld. 2020. *Revised MD17 dataset (rMD17)*. doi:10.6084/m9.figshare.12672038.v4
- [6] Taoyong Cui, Yutao Zhou, and Tong Wang. 2025. Recent advances in artificial intelligence-driven biomolecular dynamics simulations based on machine learning force fields. *Current Opinion in Structural Biology* 95 (2025), 103191. doi:10.1016/j.sbi.2025.103191
- [7] Gregory Fonseca, Igor Poltavsky, and Alexandre Tkatchenko. 2023. Force Field Analysis Software and Tools (FFAST): Assessing Machine Learning Force Fields under the Microscope. *Journal of Chemical Theory and Computation* 19, 23 (2023), 8706–8717. doi:10.1021/acs.jctc.3c00985 arXiv:https://doi.org/10.1021/acs.jctc.3c00985 PMID: 38011895.
- [8] Thorben Frank, Klaus-Robert Müller, and Oliver Unke. 2022. So3krates: Equivariant Attention for Interactions on Arbitrary Length-Scales in Molecular Systems. (2022), 29400–29413. doi:10.52202/068431-2132
- [9] Johannes Gasteiger, Shankari Giri, Johannes T. Margraf, and Stephan Günemann. 2020. Fast and Uncertainty-Aware Directional Message Passing for Non-Equilibrium Molecules. *arXiv preprint arXiv:2011.14115* (2020). arXiv:2011.14115 [cs.LG] Published at the Machine Learning for Molecules Workshop at NeurIPS 2020.
- [10] Ryosuke Jinnouchi and Saori Minami. 2025. Machine Learning Force Fields in Electrochemistry: From Fundamentals to Applications. *ACS Nano* 19, 25 (2025), 22600–22644. doi:10.1021/acsnano.5c05553 arXiv:https://doi.org/10.1021/acsnano.5c05553 PMID: 40530604.
- [11] Yi-Lun Liao, Brandon Wood, Abhishek Das, and Tess Smidt. 2024. EquiformerV2: Improved Equivariant Transformer for Scaling to Higher-Degree Representations. (2024). doi:10.48550/arXiv.2306.12059 arXiv:2306.12059 [cs.LG] Published as a conference paper at ICLR 2024.
- [12] Albert Musaelian, Simon Batzner, Anders Johansson, Lixin Sun, Cameron J. Owen, Mordechai Kornbluth, and Boris Kozinsky. 2023. Learning local equivariant representations for large-scale atomistic dynamics. *Nature Communications* 14, 1 (03 Feb 2023), 579. doi:10.1038/s41467-023-36329-y
- [13] Raul P. Pelaez, Guillem Simeon, Raimondas Galvelis, Antonio Mirarchi, Peter Eastman, Stefan Doerr, Philipp Thölke, Thomas E. Markland, and Gianni De Fabritiis. 2024. TorchMD-Net 2.0: Fast Neural Network Potentials for Molecular Simulations. *Journal of Chemical Theory and Computation* 20, 10 (2024), 4076–4087. doi:10.1021/acs.jctc.4c00253 arXiv:https://doi.org/10.1021/acs.jctc.4c00253 PMID: 38743033.
- [14] Kristof T. Schütt, Oliver T. Unke, and Michael Gastegger. 2021. Equivariant message passing for the prediction of tensorial properties and molecular spectra. *ArXiv abs/2102.03150* (2021). https://api.semanticscholar.org/CorpusID: 231839844
- [15] K. T. Schütt, H. E. Sauceda, P.-J. Kindermans, A. Tkatchenko, and K.-R. Müller. 2018. SchNet – A deep learning architecture for molecules and materials. *The Journal of Chemical Physics* 148, 24 (03 2018), 241722. doi:10.1063/1.5019779
- [16] Oliver T. Unke, Stefan Chmiela, Huziel E. Sauceda, Michael Gastegger, Igor Poltavsky, Kristof T. Schütt, Alexandre Tkatchenko, and Klaus-Robert Müller. 2021. Machine Learning Force Fields. *Chemical Reviews* 121, 16 (2021), 10142–10186. doi:10.1021/acs.chemrev.0c01111 arXiv:https://doi.org/10.1021/acs.chemrev.0c01111 PMID: 33705118.
- [17] Xujian Wang, Zeyu Sun, Yilu Zhang, Carlo Asam, Ruzhan Zhu, Wan-Lu Li, and Junmei Wang. 2026. MAPLE: a machine-learning force-field-native platform for automated reaction modeling and enzyme design. *Chem. Sci.* (2026). doi:10.1039/D6SC01279E
- [18] Yusong Wang, Tong Wang, Shaoning Li, Xinheng He, Mingyu Li, Zun Wang, Nanning Zheng, Bin Shao, and Tie-Yan Liu. 2024. Enhancing geometric representations for molecules with equivariant vector-scalar interactive message passing. *Nature Communications* 15, 1 (05 Jan 2024), 313. doi:10.1038/s41467-023-43720-2

A Training Results (Tables and Figures)

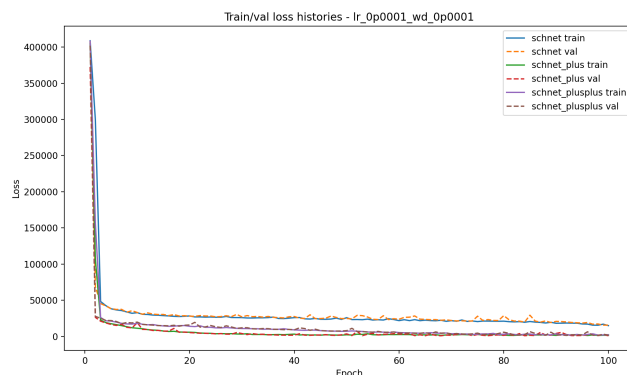


Figure 1: Train/val loss history for the three models on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.0001).

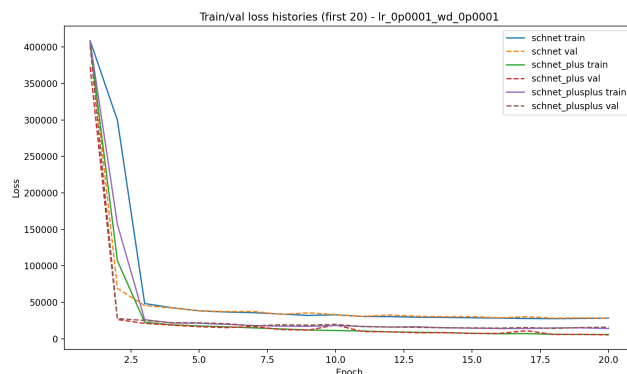


Figure 2: Train/val loss history (first 20 epochs) for the three models on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.0001). Note the faster decrease in train/val loss for SchNet+ and SchNet++.

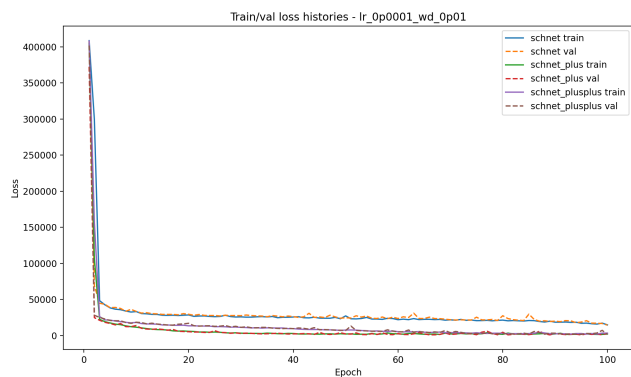


Figure 3: Train/val loss history for the three models on rMD17 aspirin (learning rate = 0.0001, weight decay = 0.01).

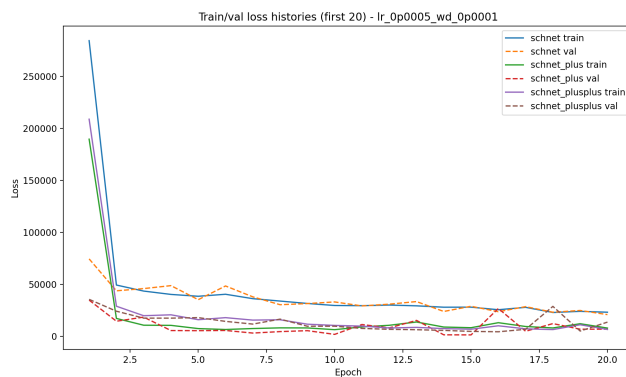


Figure 6: Train/val loss history (first 20 epochs) for the three models on rMD17 aspirin (learning rate = 0.0005, weight decay = 0.0001). Note the faster decrease in train/val loss for SchNet+ and SchNet++.

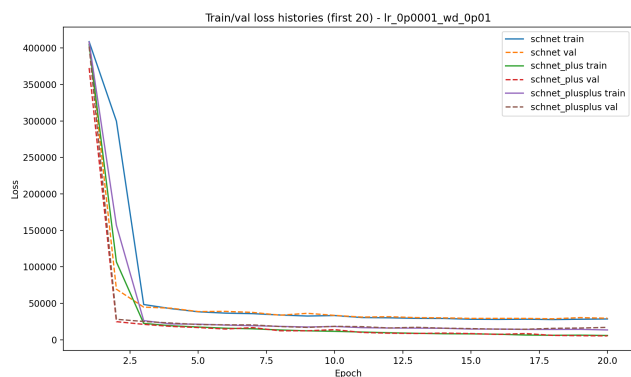


Figure 4: Train/val loss history (first 20 epochs) for the three models on rMD17 aspirin (learning rate = 0.0001, weight decay = 0.01). Note the faster decrease in train/val loss for SchNet+ and SchNet++.

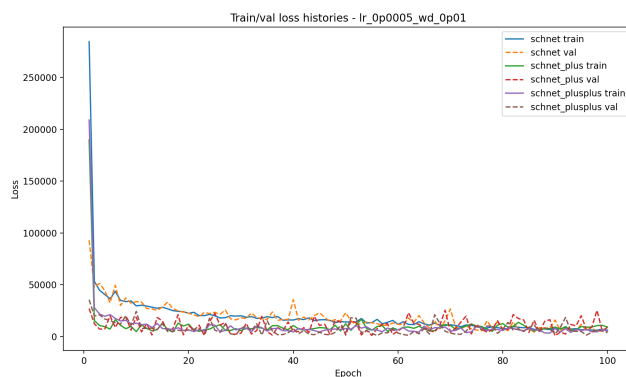


Figure 7: Train/val loss history for the three models on rMD17 aspirin (learning rate = 0.0005, weight decay = 0.01).

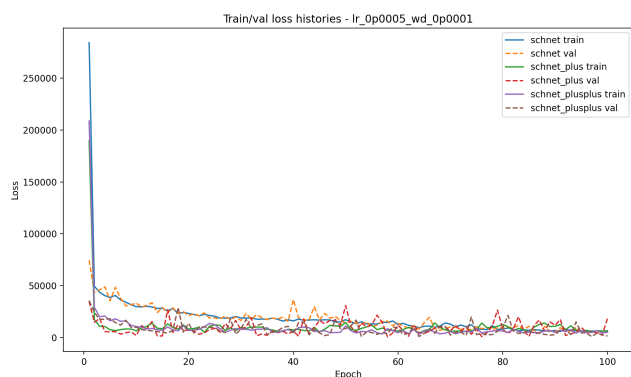


Figure 5: Train/val loss history for the three models on rMD17 aspirin (learning rate = 0.0005, weight decay = 0.0001).

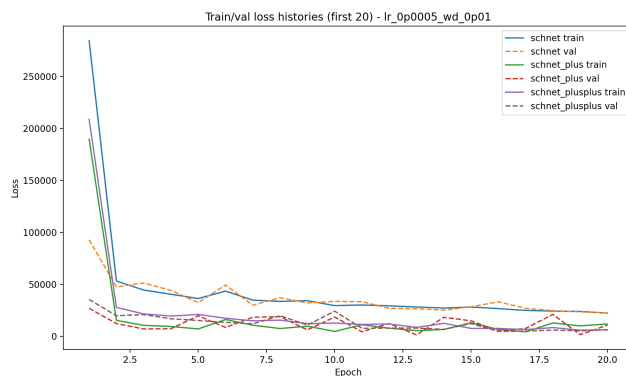


Figure 8: Train/val loss history (first 20 epochs) for the three models on rMD17 aspirin (learning rate = 0.0005, weight decay = 0.01). Note the faster decrease in train/val loss for SchNet+ and SchNet++.

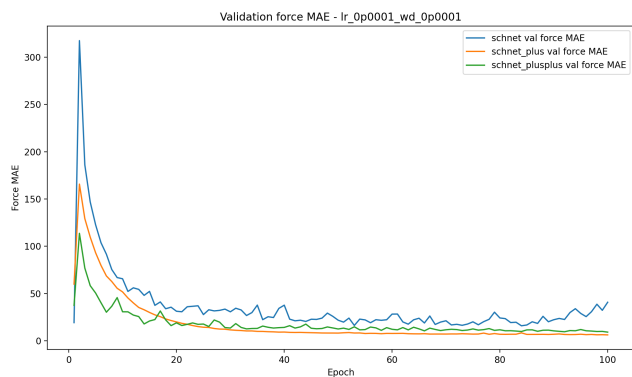


Figure 9: Force MAE (Mean Absolute Error) on validation dataset during the training epochs of the three models on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.0001).

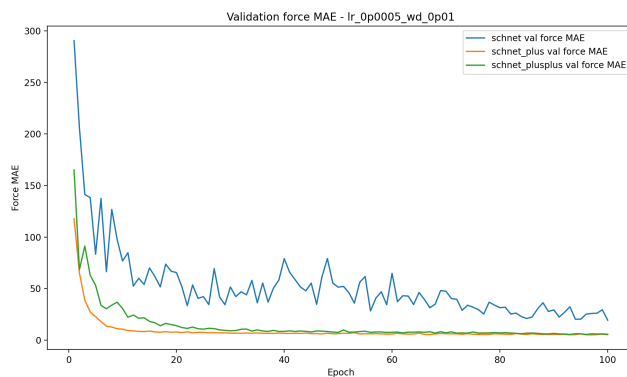


Figure 12: Force MAE (Mean Absolute Error) on validation dataset during the training epochs of the three models on rMD17 aspirin (learning rage = 0.0005, weight decay = 0.01).

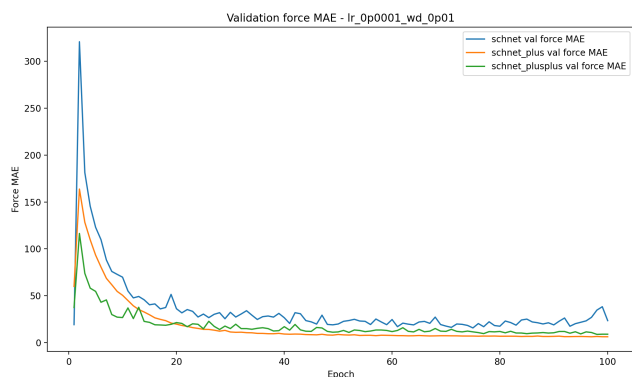


Figure 10: Force MAE (Mean Absolute Error) on validation dataset during the training epochs of the three models on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.01).

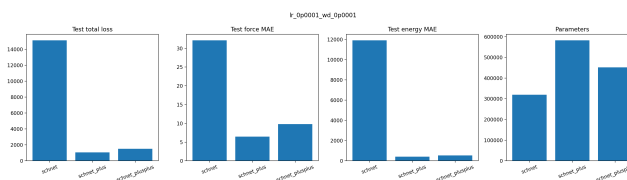


Figure 13: Test MAEs comparison for the three models and their parameters count on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.0001).

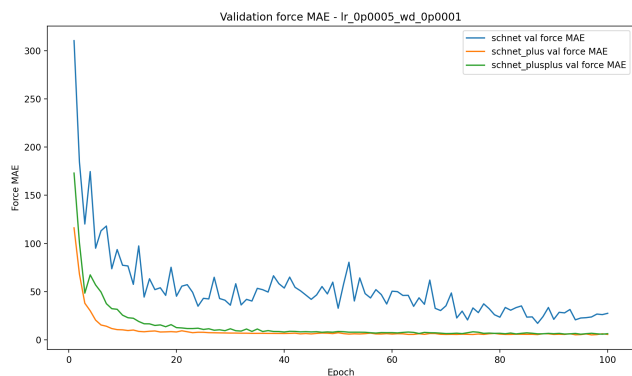


Figure 11: Force MAE (Mean Absolute Error) on validation dataset during the training epochs of the three models on rMD17 aspirin (learning rage = 0.0005, weight decay = 0.0001).

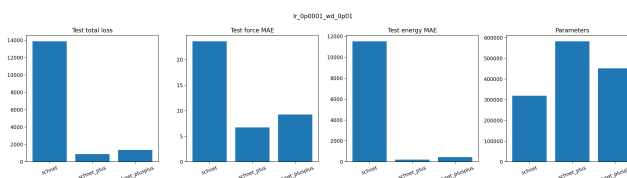


Figure 14: Test MAEs comparison for the three models and their parameters count on rMD17 aspirin (learning rage = 0.0001, weight decay = 0.01).

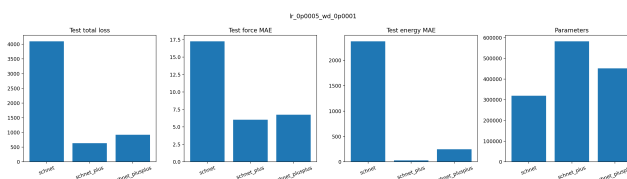


Figure 15: Test MAEs comparison for the three models and their parameters count on rMD17 aspirin (learning rage = 0.0005, weight decay = 0.0001).

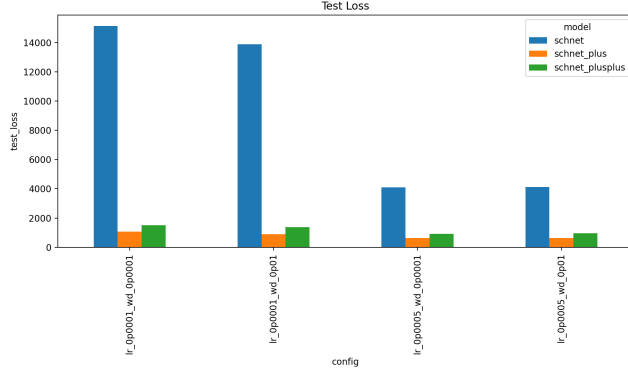


Figure 16: Cross configuration total test loss for the three models.

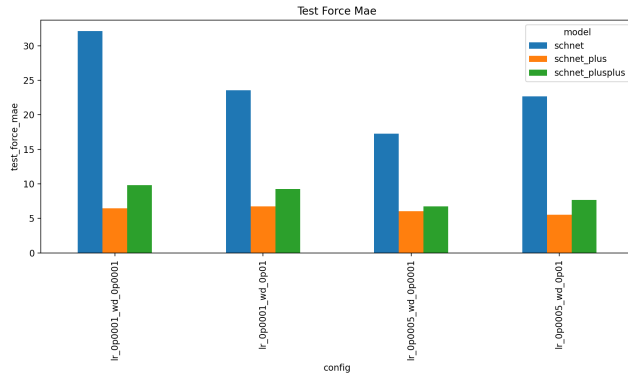


Figure 17: Cross configuration forces test loss for the three models.

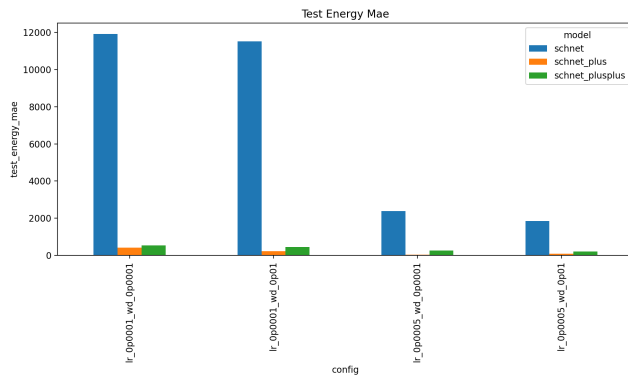


Figure 18: Cross configuration Energy test loss for the three models.

We further define an efficiency score which evaluates how much the test loss of the model has decreased with respect to the increase

in the number of parameters (having SchNet as baseline). The efficiency score is defined as follows:

$$efscore(m) = \frac{\frac{TL(SchNet) - TL(m)}{TL(SchNet)}}{\frac{P(m) - P(SchNet)}{P(SchNet)}} \quad (11)$$

Where $TL(m)$ is the Total Test Loss, and $P(m)$ is the number of parameters for model m . This metric calculates the ratio of decrease in loss with respect to the ratio of increase in the number of parameters. Any number higher than 1 suggests that the model was able to decrease the loss more than it increased the number of parameters compared to the baseline model.

Table 1: Efficiency Score for SchNet+ and SchNet++ (Averaged over Configurations)

SchNet+	SchNet++	Metric
0.89	0.84	Relative Total Error Improvement
0.82	0.41	Relative Parameter Increase
1.09	2.05	Efficiency Score

As shown by the table, SchNet+ and SchNet++ both remarkably improve the total loss. However, SchNet++ uses much less number of parameters as SchNet+ while achieving roughly the same loss over the test dataset.

B Number of Parameters

In this section, we do a mathematical analysis on the number of parameters of SchNet, SchNet+, SchNet++. Then, we will show that as our machine learning models grow, their sizes become proportional to each other, meaning the proportion of SchNet+ and SchNet++'s size to SchNet is $O(1)$.

B.1 SchNet Parameters

The general workflow for calculating the number of parameters is as follows:

$$P(\text{SchNet}) = m \times (P(\text{Interaction Layer}) + P(\text{Update}) + P(\text{Atomic Embedding}) + P(\text{Energy Head})) \quad (12)$$

Furthermore, we assume that the number of hidden features (n) is constant over the interaction layers (m). Therefore, the number of parameters for SchNet would be:

$$m \times \left([n_{rbf} \times n + (n^2 + n) + (n^2 + n)] + [n^2 + n + n^2 + n] + [118 \times n] + [n^2 + n + n + 1] \right) \quad (13)$$

n_{rbf} is the number of radial basis functions. Each layer transforms the pairwise distance matrix to vectors with same size as hidden features using a simple 2 layered MLP. Also, each atom embedding (or hidden embedding) is transformed to another vector with the same size using another single layer MLP. Initially, each atom has an embedding of size n , resulting into a matrix of initial learnable embeddings for each atomic number (hence the term $118 \times n$). Lastly, the Energy Head is another 2 layer MLP getting vectors of size n as input and giving one scalar as output.

B.2 SchNet+ and SchNet++

The general workflow for calculating the number of parameters for SchNet+ is as follows:

$$P(\text{SchNet+}) = m \times (P(\text{Interaction Layer}) + P(\text{Update}) + P(\text{Fuse})) + P(\text{Atomic Embedding}) + P(\text{Energy Head}) \quad (14)$$

In addition to the previous structure, each interaction layer now has an additional fuse MLP which integrates the different embeddings proposed by different aggregations. Hence, the number of parameters for SchNet+ would be:

$$m \left([n_{rbf} \times n + (n^2 + n) + (n^2 + n)] + [n^2 + n + n^2 + n] + [4n^2 + n] \right) + [118 \times n] + [n^2 + n + n + 1] \quad (15)$$

Similarly, for SchNet++ we would have:

$$P(\text{SchNet++}) = m \left(P(\text{Interaction Layer}) + P(\text{Update}) + P(\text{Fuse}) + P(\text{f-mean}) \right) + P(\text{Atomic Embedding}) + P(\text{Energy Head}) \quad (16)$$

The only thing, additional to SchNet+ is the α_d factor inside the log-sum-exp function which adds n learnable parameters to each layer.

Thus:

$$m \left([n_{rbf} \times n + (n^2 + n) + (n^2 + n)] + [n^2 + n + n^2 + n] + [2n^2 + n] + n \right) + [118 \times n] + [n^2 + n + n + 1] \quad (17)$$

B.3 Comparison

The total number of parameters for each architecture can be summarized as follows:

$$\text{SchNet}(n, m) = (4m + 1)n^2 + (5m + m \times n_{rbf} + 120)n + 1 \quad (18)$$

$$\text{SchNet+}(n, m) = (8m + 1)n^2 + (6m + m \times n_{rbf} + 120)n + 1 \quad (19)$$

$$\text{SchNet++}(n, m) = (6m + 1)n^2 + (7m + m \times n_{rbf} + 120)n + 1 \quad (20)$$

The proportion between parameters of SchNet+ and SchNet is:

$$\lim_{m, n \rightarrow \infty} \frac{\text{SchNet+}(n, m)}{\text{SchNet}(n, m)} = \frac{(8m + 1)n^2 + (6m + m \times n_{rbf} + 120)n + 1}{(4m + 1)n^2 + (5m + m \times n_{rbf} + 120)n + 1} = \frac{8mn^2}{4mn^2} = 2 \quad (21)$$

Moreover, the proportion between parameters of SchNet++ and SchNet is:

$$\lim_{m, n \rightarrow \infty} \frac{\text{SchNet++}(n, m)}{\text{SchNet}(n, m)} = \frac{(6m + 1)n^2 + (7m + m \times n_{rbf} + 120)n + 1}{(4m + 1)n^2 + (5m + m \times n_{rbf} + 120)n + 1} = \frac{6mn^2}{4mn^2} = 1.5 \quad (22)$$

Note that, each SchNet model can be extended to SchNet+ or SchNet++, and as our SchNet grows, the extended SchNet+ and SchNet++ also grow but the size of their parameters grow with a factor of 2-1.5 (respectively).

C Source Code

Source code for this project is available at:
<https://github.com/Amirarsalan-sn/SchNetPlus>

D Acknowledgment

We acknowledge the collaborative usage of AI in the processes of project development and writing the report.